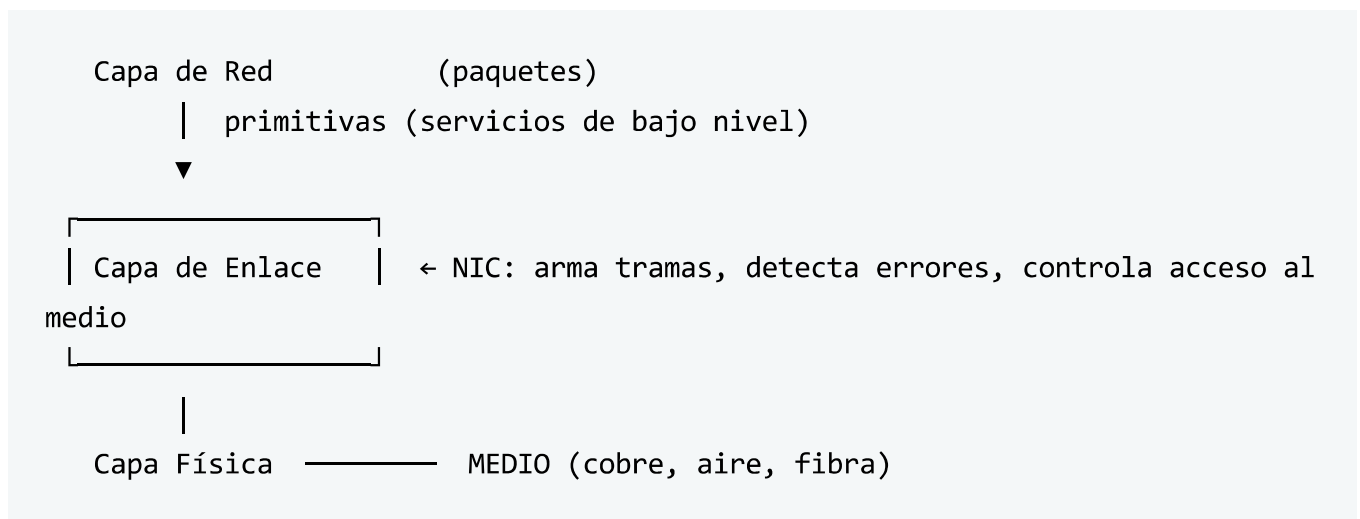


Resumen de Clase 18/09/2026 Redes Emily Sánchez Orozco - 2021067314

1. ¿Dónde está y qué hace?

La capa de enlace vive en el punto donde la **tarjeta de red (NIC)** se conecta con el **medio físico**. Usa los servicios de la capa física para enviar y recibir bits, y le ofrece servicios a la capa de red.



Funciones principales

Función	Qué significa
Servicios a la capa de red	Se exponen como primitivas
Manejo de errores	Sobre todo detección ; la corrección es limitada
Regulación del canal	Evitar que alguna estación se quede "con hambre" (sin poder transmitir)
Encapsulamiento	El paquete de red se vuelve el payload de la trama
Manejo de tramas	La trama es la unidad de envío local

2. Las tramas son locales

Una trama solo existe **dentro de una LAN**.

PC A —trama→ Switch —trama→ Router —(trama NUEVA)→ otro router →
...
|
desempaqueta la trama,
lee el paquete IP y decide
hacia dónde enviarlo

- Tráfico local: el switch lee el destino de la trama y la entrega al puerto correcto.
- Tráfico hacia fuera: la trama llega al router, el router **destruye** la trama original, saca el paquete y genera una **trama nueva** para el siguiente salto.

Estructura de una trama

Header	Payload (paquete red)	Trailer (CRC)
--------	-----------------------	---------------

3. Modos de operación

Modo	Cómo funciona	Ejemplo	¿Por qué?
Sin conexión, sin confirmación	Envío y no me importa si llegó. Sin establecer ni liberar conexión	Ethernet	El cobre es muy confiable, casi no se pierden tramas; pedir confirmación sería desperdicio
Sin conexión, con confirmación	Cada trama se confirma (ACK); si vence un temporizador, se reenvía	Wi-Fi	El aire tiene muchos errores; requiere muchos <i>retries</i> y <i>timeouts</i>
Orientado a conexión, con confirmación	Tramas numeradas y ordenadas; se usan ACK, NACK y temporizadores	—	Garantiza orden y entrega completa

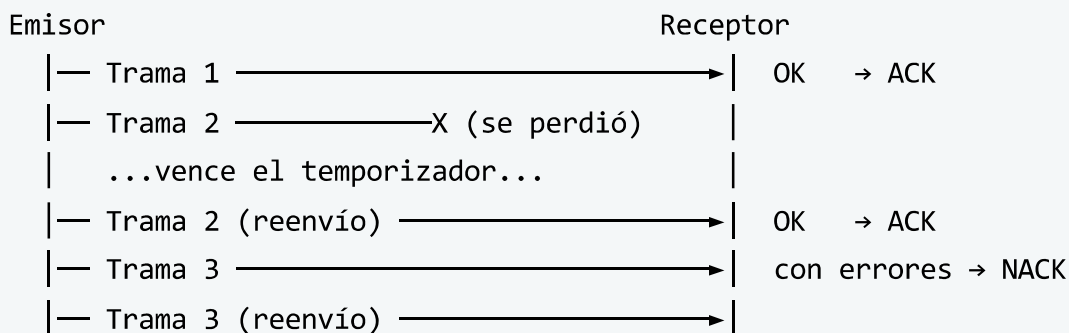
Parada y espera (stop-and-wait) y su costo

Enviar una trama y quedarse esperando el ACK desperdicia el canal.

Analogía del bus: un bus va de Cartago a San José (1 h) y al llegar mandan a alguien en moto a avisar que llegó bien (10 min). Durante 1 h 10 min la calle solo sirvió para un viaje. Hubiera sido mejor mandar **varios buses seguidos**.

Por eso conviene enviar **grupos de tramas** una tras otra para aprovechar el canal.

Ciclo de vida de una trama (orientado a conexión)



Idea clave: la confiabilidad se puede poner en cualquier capa

Aunque todas las capas de abajo sean "no confiables" (ej. Ethernet + UDP), una aplicación puede garantizar entrega:

```
# Idea de un programa sobre UDP
enviar(dato)
esperar_ack(timeout=30)  # si no llega confirmación...
reintentar(max=5)        # ...reenvío, hasta 5 veces
```

Entre más arriba se implemente la confiabilidad, **peor rendimiento**, porque hay que desempaquetar todo hasta esa capa para decidir si reenviar.

4. Entramado (framing)

Un paquete de red se parte en tramas, y cada trama viaja como una secuencia de bytes (ej. `0xFA0x35 0x41 ...`). El receptor necesita saber **dónde empieza y termina** cada trama, sabiendo que cualquier parte (header, payload, trailer) puede dañarse.

4.1 Conteo de bytes

El primer byte indica cuántos bytes tiene la trama.

Correcto: `[5][x x x x] [3][x x] [8][x x x x x x x]`
 ↑ incluye el propio contador

Error (5 → 9):
 `[9][x x x x 3 x x 8 x] ...`
 └ lee 9 bytes: mezcla dos tramas y pierde la sincronía

No sirve sola: un error en el contador arruina todo lo que sigue.

4.2 Byte bandera con relleno de bytes (byte stuffing)

Un byte especial (**FLAG**) marca inicio y fin. Si el dato contiene FLAG o ESC, se le antepone un byte de escape (**ESC**), igual que `\` en un string.

Datos originales	Se envía
A FLAG B	A ESC FLAG B
A ESC B	A ESC ESC B

Trama completa: FLAG | datos con relleno | FLAG

Si se daña una bandera, el receptor eventualmente ve dos banderas seguidas y detecta el error.

4.3 Bandera con relleno de bits (bit stuffing) — HDLC

- La trama inicia y termina con la bandera `0x7E = 01111110` (elegido porque estadísticamente es poco común).
- Cada vez que el emisor ve **cinco 1 seguidos**, inserta un `0`.
- El receptor, al ver cinco `1` seguidos de un `0`, descarta ese `0`.

```
Original:  011011111111111111110010
Enviado:   011011111 0 11111 0 11111 0 10010
           = 011011111011111011111010010
```

Beneficio extra: sincronización. Emisor y receptor no comparten reloj; muchos bits iguales seguidos hacen que se desincronicen. El `0` insertado da una transición que permite "reajustar el reloj". USB usa esta idea (con una variante del conteo).

5. Detección y corrección de errores

5.1 Conceptos base

- Se necesitan **números de trama** para saber qué reenviar.
- Detectar/corregir requiere **información redundante**, que cuesta ancho de banda (y dinero). Ej: enviar cada trama dos veces convierte un enlace de 400 Mbps en uno de 200 Mbps efectivos.
- Dos familias:
 - Códigos de corrección:** detectan y deducen qué bit arreglar.
 - Códigos de detección:** solo avisan que hubo error.
- En la capa de enlace se hace **principalmente detección**.

5.2 Errores en ráfaga

Un bit es tan "pequeño" en el espectro electromagnético que una interferencia casi nunca daña uno solo: normalmente daña **un grupo de bits cercanos**.

```
Enviado:  1 0 1 1 0 1 1 0 1 0
Recibido: 1 0 1 0 1 0 0 0 1 0
          ^^^^^^  ← ráfaga
```

5.3 Distancia de Hamming

Cantidad de bits que hay que voltear para convertir una palabra válida en otra palabra válida. Se calcula con **XOR** y contando los **1**.

```
10001001
⊕ 10110001
= 00111000 → 3 unos → distancia de Hamming = 3
```

Si una ráfaga convierte una palabra válida en otra válida, el receptor **no puede notar** el error.

5.4 Tasa de código

tasa = bits de mensaje / bits totales enviados

- Medios confiables (cobre): pocos bits de verificación → tasa alta.
- Medios con errores (wireless): más redundancia → tasa más baja.

6. Bit de paridad

Se cuentan los **1** de la palabra. Si la cantidad es **impar**, el bit de paridad es **1**; si es **par**, es **0** (el total queda par).

Datos	# de unos	Paridad	Enviado
10110000	3 (impar)	1	101100001
11001100	4 (par)	0	110011000

Tasa: **8/9**.

Problema: si se voltean **dos** bits, la paridad sigue siendo correcta y el error pasa desapercibido. Solo sirve en medios muy confiables.

7. Código de Hamming (detecta y puede corregir 1 bit)

Reglas

1. Numerar los bits del mensaje codificado de izquierda a derecha **empezando en 1**.
2. Posiciones que son **potencias de 2** (1, 2, 4, 8, ...) → bits de verificación.
3. El resto → bits de datos.
4. Se escribe como **Hamming (n, m)**: n = total de bits, m = bits de datos.
5. Cada posición de datos se escribe como suma de potencias de 2; esas potencias indican qué bits de verificación la cubren.
 - Ej: $10 = 2 + 8$ → el bit 10 lo verifican P2 y P8.

Ejemplo: Hamming (11, 7), mensaje 1000001

Quién verifica a quién

Posición de dato	Suma de potencias	La verifican
3	1 + 2	P1, P2
5	1 + 4	P1, P4
6	2 + 4	P2, P4
7	1 + 2 + 4	P1, P2, P4
9	1 + 8	P1, P8
10	2 + 8	P2, P8
11	1 + 2 + 8	P1, P2, P8

Colocar datos y calcular paridades

Posición:	1	2	3	4	5	6	7	8	9	10	11
Tipo:	P1	P2	d	P4	d	d	d	P8	d	d	d
Datos:	—	—	1	—	0	0	0	—	0	0	1

Bit	Cubre	Valores	# unos	Valor
P1	3,5,7,9,11	1,0,0,0,1	2 (par)	0
P2	3,6,7,10,11	1,0,0,0,1	2 (par)	0
P4	5,6,7	0,0,0	0 (par)	0
P8	9,10,11	0,0,1	1 (impar)	1

Palabra codificada enviada: 0 0 1 0 0 0 0 1 0 0 1 → 00100001001

Tasa: 7/11 (7 de datos, 4 de redundancia).

En el receptor: detectar y corregir

Se recalcula la paridad de cada grupo **incluyendo** su bit de verificación. Si todo llegó bien, todos dan 0.

Supongamos que el **bit 5** llega volteado (0 → 1): 00101001001

Grupo	Bits (posiciones)	Valores	Resultado
P1	1,3,5,7,9,11	0,1,1,0,0,1	impar → 1 x
P2	2,3,6,7,10,11	0,1,0,0,0,1	par → 0
P4	4,5,6,7	0,1,0,0	impar → 1
P8	8,9,10,11	1,0,0,1	par → 0

Síndrome = suma de los grupos que fallaron = 1 + 4 = 5 → el bit dañado es el 5.

Visto como binario (P8 P4 P2 P1): 0 1 0 1 = $0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$.

Corrección: se complementa el bit 5 (1 → 0).

⚠ Hamming básico **corrige 1 bit**; con 2 o más bits dañados ya no puede ubicar el error correctamente. Por eso en esta capa se enfatiza la **detección**.

8. Paridad bidimensional (filas y columnas)

Se ve la información como una cuadrícula y se calcula paridad por **fila** y por **columna** (aquí paridad par).

datos		paridad fila
1 0 1 1		1
0 1 1 0		0
1 1 0 0		0
0 1 0 1		0
par. col: 0 1 0 0		1

Si llega volteado el bit de la fila 2, columna 3 (0110 → 0100):

1 0 1 1		1	
0 1 [0] 0		0	← falla la fila 2
1 1 0 0		0	
0 1 0 1		0	
0 1 0 0			← falla la columna 3

La intersección da la **coordenada** del error → se puede voltear y corregir. Detecta mejor las ráfagas que la paridad simple, pero tampoco se usa mucho en esta capa.

9. Suma de verificación (checksum)

Idea: el emisor suma los datos y envía la suma al final; el receptor vuelve a sumar y compara.

Datos: 10, 13, 40, 13, 12

Suma: 10+13 = 23 → +40 = 63 → +13 = 76 → +12 = 88

Se envía: 10 13 40 13 12 | 88

Receptor: suma lo recibido → ¿da 88? sí → sin error / no → error

En la implementación real, se envía el **complemento** de la suma (a veces complemento a 1), de modo que al sumar todo, incluido el checksum, el resultado **se desborda y da 0**.

10. CRC — Código de Redundancia Cíclica

Es lo que usan **Ethernet y wireless** (va en el trailer de la trama).

- El **polinomio generador $G(x)$** lo define el **protocolo**; no se elige al configurar una red.
- Si $G(x)$ tiene grado r (es decir, $r + 1$ bits), se agregan **r ceros** al mensaje.
- Se hace **división módulo 2** = restas con **XOR** → circuitos muy simples y rápidos.

Algoritmo

1. Agregar r ceros al final del mensaje.
2. Dividir módulo 2 entre $G(x)$.
3. El residuo (r bits) reemplaza esos ceros → palabra codificada.
4. El receptor divide la palabra recibida entre $G(x)$ **sin agregar ceros**. Residuo **0** → sin error.

Ejemplo

- Mensaje: **11101001**
- Generador: **10011** (5 bits → grado 4 → se agregan **4 ceros**)
- Dividendo: **11101001 0000**

```
11101001 0000
10011                ← XOR
-----
1110 0               baja un bit → 11100
1001 1
-----
1111 0               baja → 11110
1001 1
-----
1101 1               baja → 11011
1001 1
-----
1000 0               baja → 10000
1001 1
-----
0011 0               baja → 00110 (empieza en 0: no se resta)
0110 0               baja → 01100 (empieza en 0: no se resta)
1100 0               baja → 11000
1001 1
-----
1011                ← RESIDUO (grado 3 < 4, ya no se puede dividir)
```

Palabra enviada: $11101001 + 1011 = 111010011011$

Receptor: divide 111010011011 entre $1011 \rightarrow$ residuo 0000  (no hubo error).

¿Por qué CRC?

- Detecta muy bien los errores, incluidas las ráfagas y los casos que "engañan" a la paridad.
- Solo usa XOR $\rightarrow$ hardware más simple y rápido que el de una suma de verificación.

--

Avisos del curso

- **Proyecto 1:** entrega movida al **28/09 antes de medianoche**.
- **Evaluación escrita del proyecto:** **29/09** (mismo formato de siempre).